



Engenharia de software e modelagem II

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

Sobre o que iremos falar ?



- **Definições sobre arquitetura e seus padrões**



Etapas do projeto



1. Refinamento do Modelo de Análise e requisitos

2. Caso de uso



Método MoSCoW

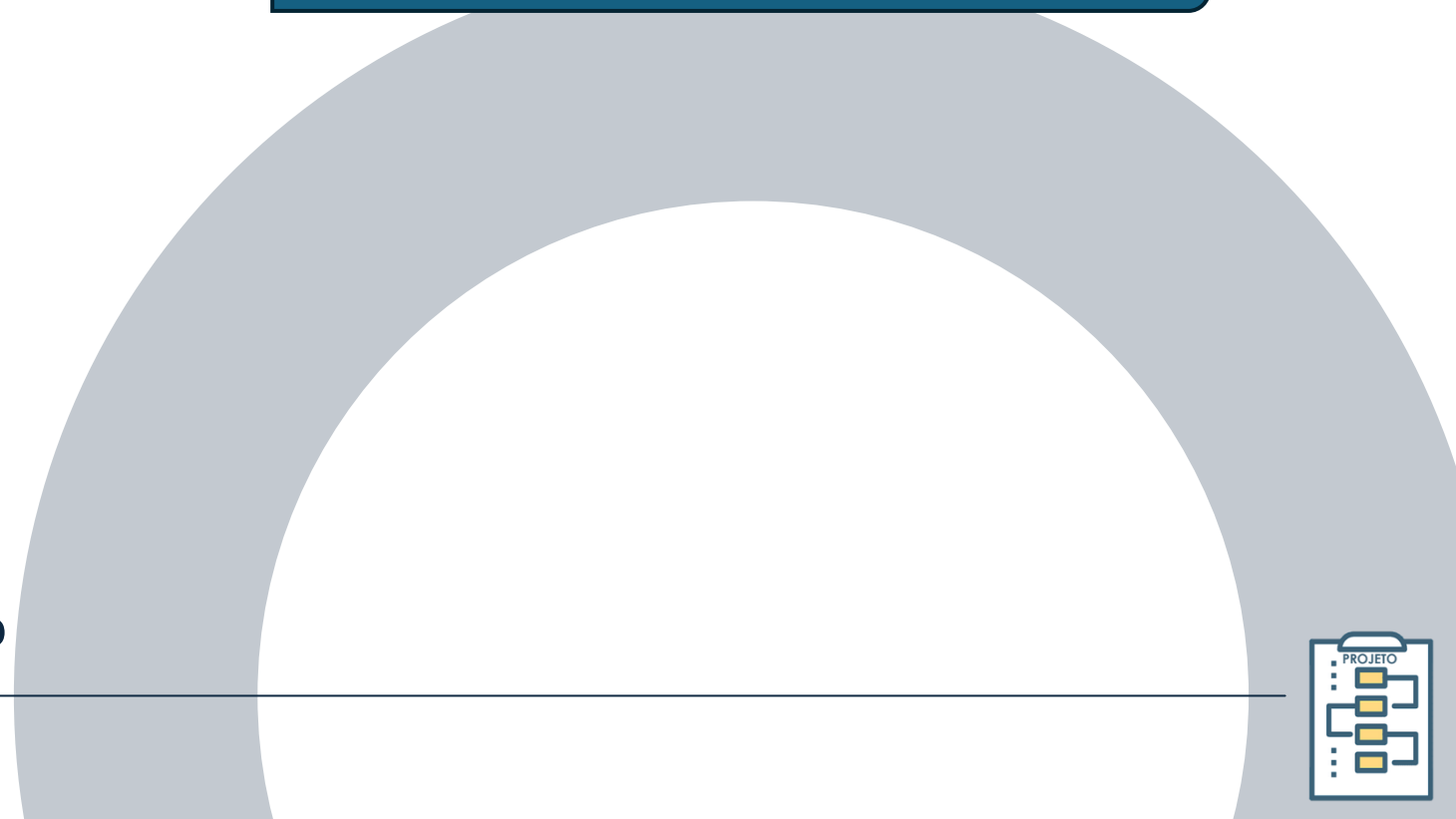
3. Modelagem de Classes

4. Definição da Arquitetura

5. Modelagem de Interações

6. Definição de Interfaces

7. Aplicação de Padrões de Projeto



O que é arquitetura de software

- Segundo a ISSO/IEC/IEEE 42010:2022

"estrutura fundamental ou esqueleto de um sistema de software, que define seus componentes, suas relações e seus princípios de projeto e evolução."

- Similar à arquitetura física: normas e técnicas para construção, mas voltada ao mundo do software.



Por que padrões arquiteturais são importantes?

- São soluções reutilizáveis e consolidadas para desafios comuns em desenvolvimento.
- Benefícios incluem:
 - Flexibilidade e escalabilidade
 - Facilidade de manutenção e evolução
 - Segurança e melhor desempenho
 - Redução de custos e riscos



Modelo de distribuição de software

- **SaaS (Software as a Service):** aplicações baseadas na nuvem, acessadas como serviço.
- **On-Premises:** softwares instalados e executados localmente na infraestrutura do cliente.
-  **Por que entender esses modelos é importante?**
 - Impacta custos e investimentos da empresa
 - Define quem é responsável por manutenção e segurança
 - Influencia na escalabilidade e flexibilidade do sistema

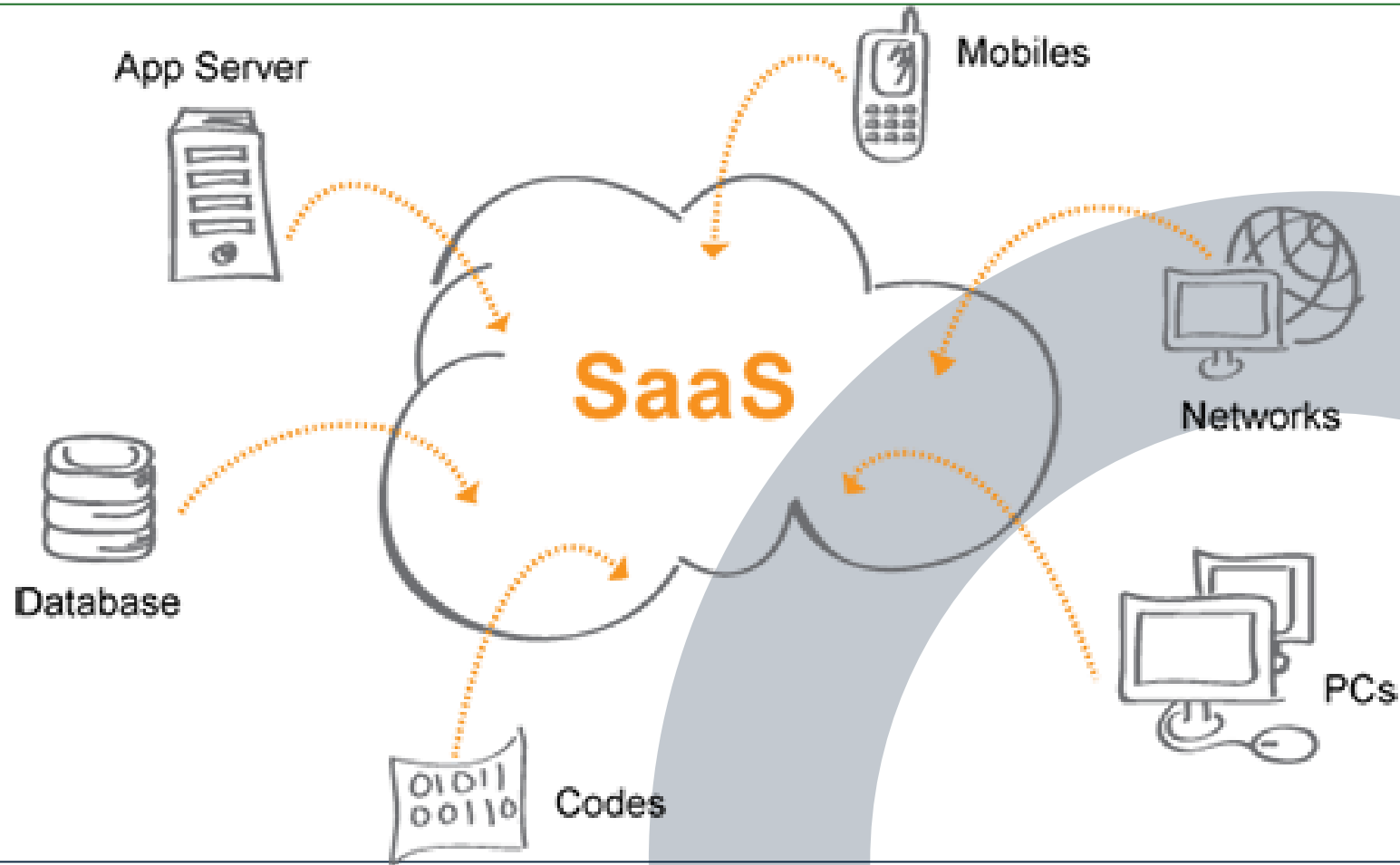


SaaS (Software as a Service)

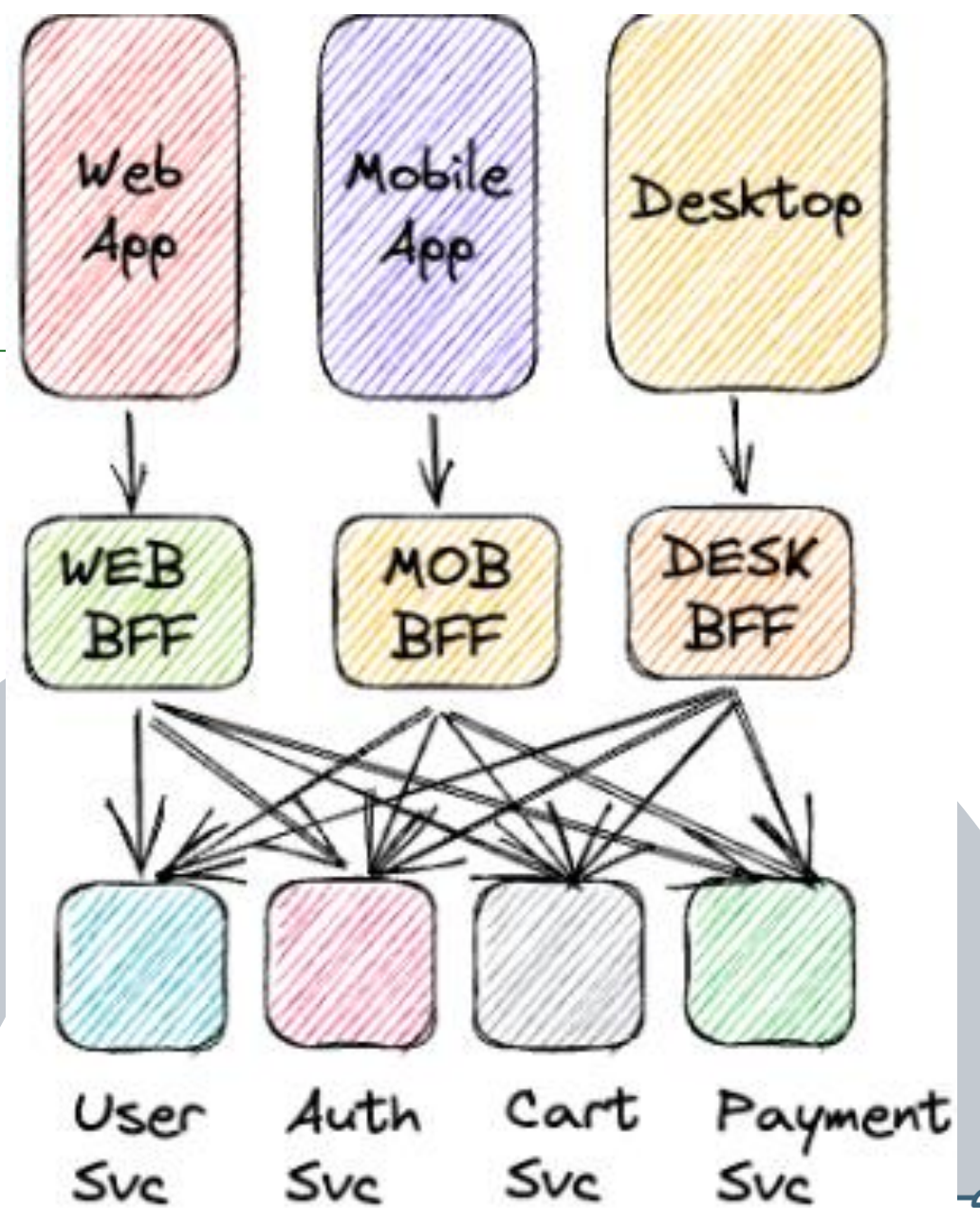
- **Definição:** Aplicação hospedada na nuvem, acessada via internet como um serviço.
- **Características:**
 - Acesso por assinatura (mensal/anual)
 - Hospedagem e manutenção feitas pelo fornecedor
 - Escalabilidade rápida
 - Atualizações automáticas
 - Acessível de qualquer dispositivo com internet



SaaS (Software as a Service)



SaaS (Software as a Service)

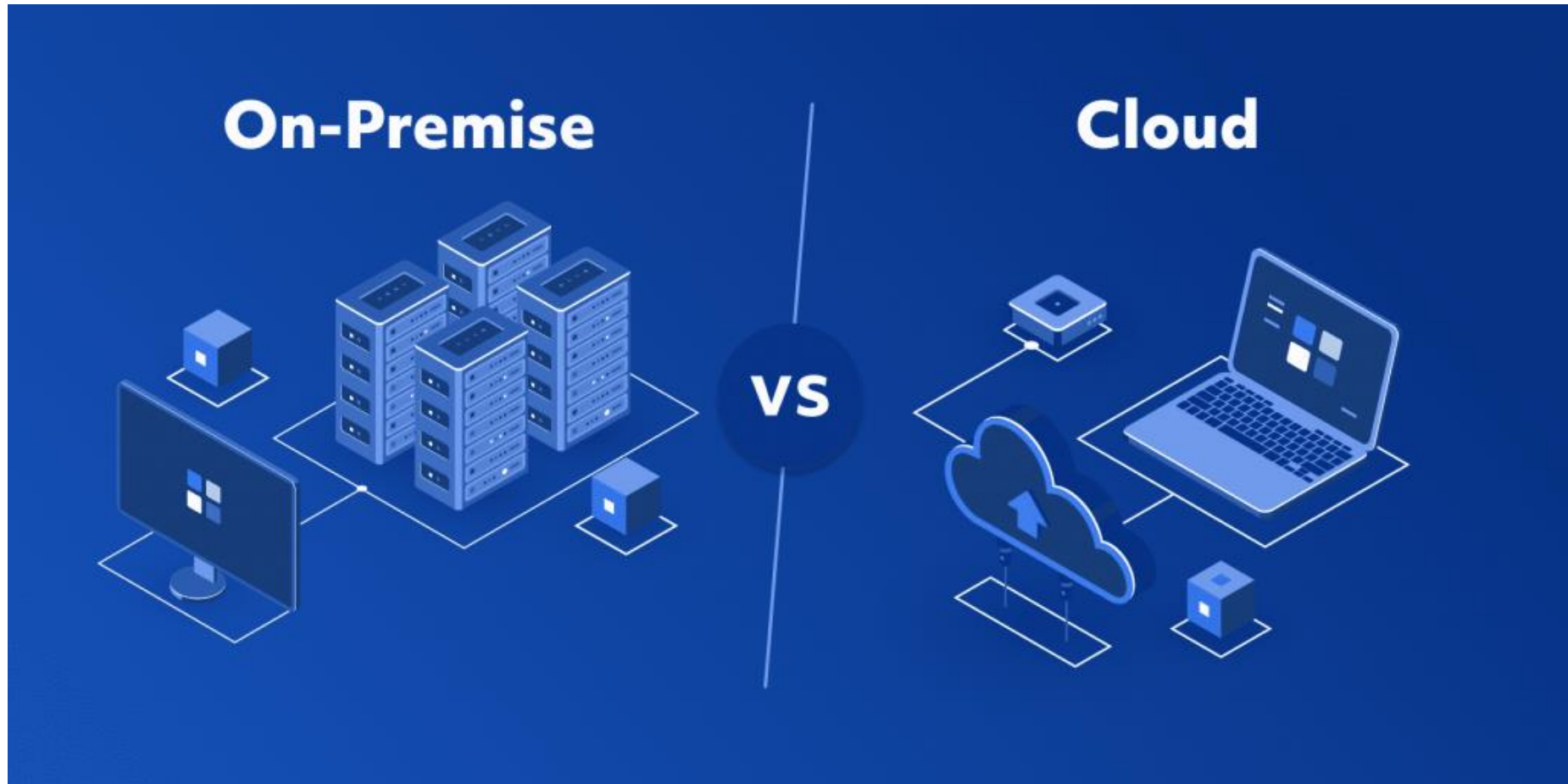


On-Premises (Instalação Local)

- **Definição:** Aplicação instalada e executada localmente na infraestrutura do cliente.
- **Características:**
 - Licença perpétua ou compra única
 - Hospedagem em servidores ou máquinas do cliente
 - Controle total sobre dados e infraestrutura
 - Atualizações dependem da equipe interna
 - Maior investimento inicial em hardware e suporte



On-Premises (Instalação Local)

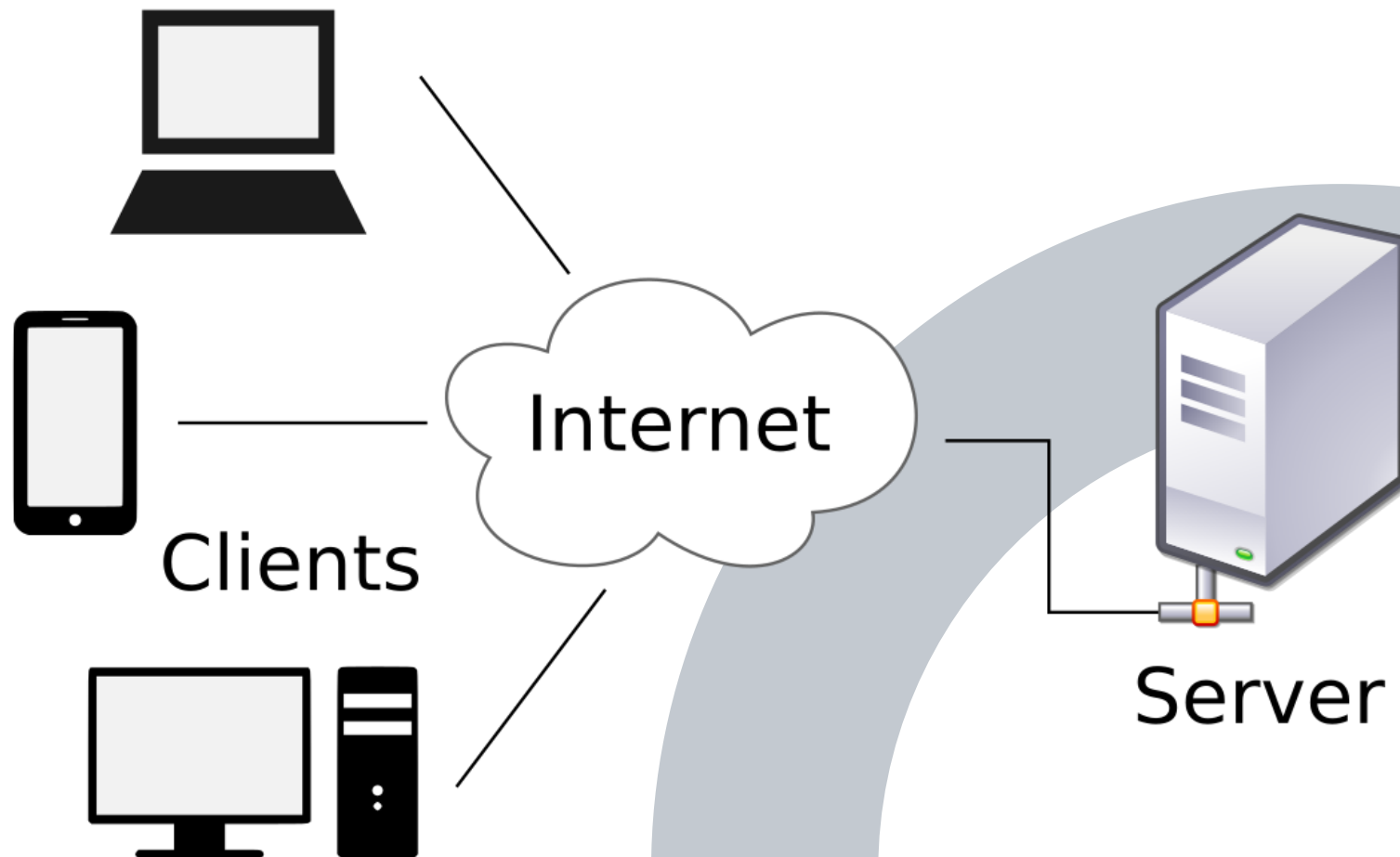


Arquitetura cliente-servidor

- Popular nos anos 1990, separava a aplicação em:
 - Cliente: interface e parte da lógica da aplicação
 - Servidor: geralmente o banco de dados (SQL)
- Limitações quanto a lógica de domínio complexa ou duplicação de código no cliente.



Arquitetura cliente-servidor



Arquitetura Orientada a Serviços (SOA)

- **Organiza o sistema em serviços independentes e comunicantes (stateless), conectados via interfaces bem definidas.**
- Exemplo: clínica médica com serviços para recepção, marcação de consultas, pedidos de exames etc.
- Requer mais complexidade e investimento, mas traz escalabilidade.



Arquitetura Orientada a Serviços (SOA)

PLATFORM

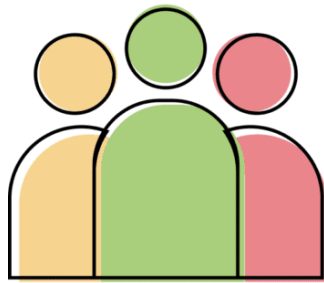


Integration



SOA

Service Oriented Architecture



PEOPLE



PROCESS



Exemplo: Sistema de E-commerce utilizando SOA

- Serviços
 - Serviço de Clientes
 - Serviço de Produtos
 - Serviço de Pedidos
 - Serviço de Pagamento
 - Serviço de Entrega



Por que isso é SOA?

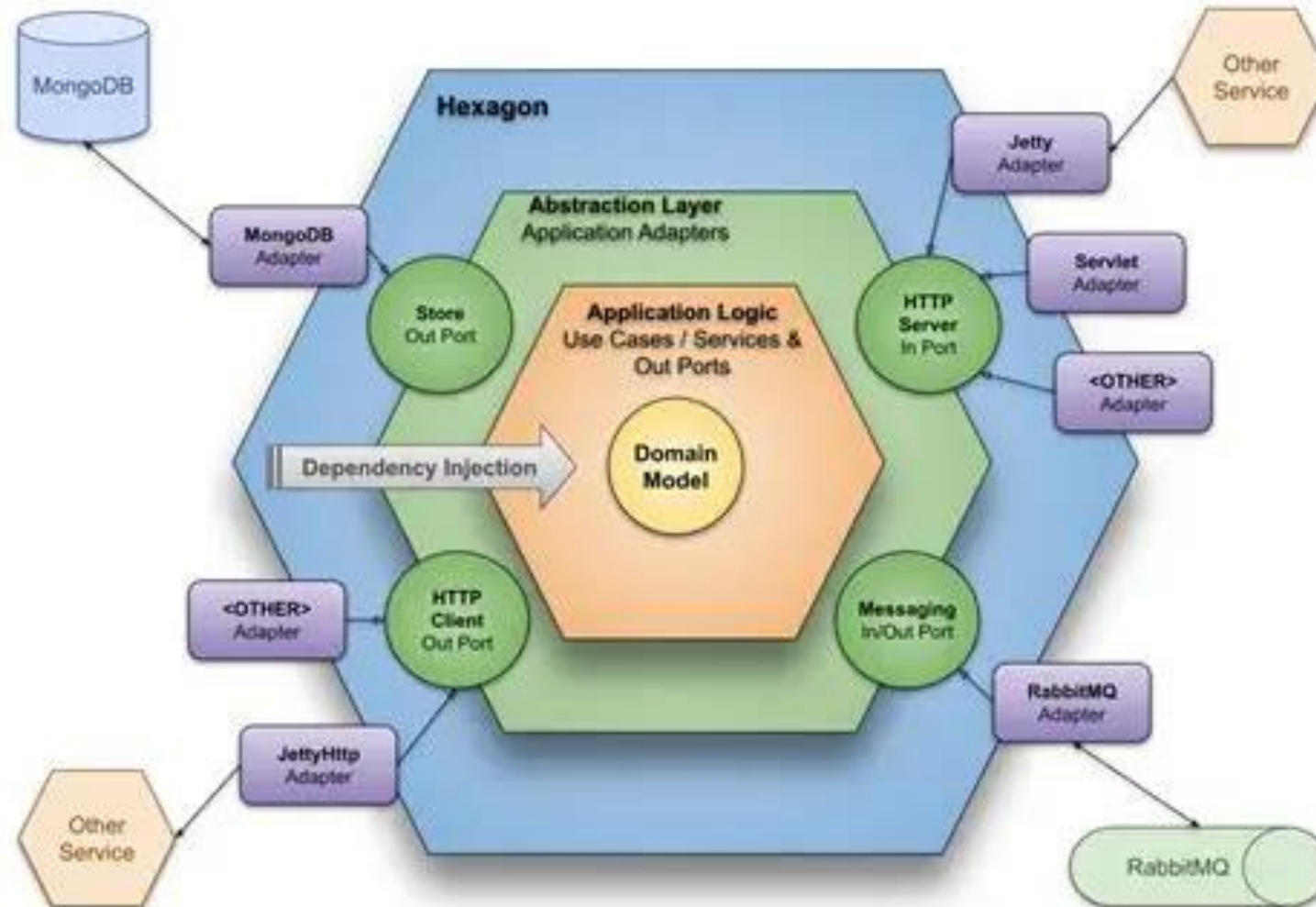
- Cada serviço possui uma **responsabilidade específica** e disponibiliza suas funcionalidades por meio de uma **interface bem definida**.
 - Diferentes sistemas podem reutilizar os mesmos serviços.
 - **Exemplo:** o Serviço de Pagamento pode ser utilizado pelo **site**, pelo **aplicativo mobile** e até por outros sistemas da empresa.
-

Arquitetura Hexagonal (Ports and Adapters)

- Desacopla lógica de negócio e implementação:
 - **Portas:** interfaces da lógica de negócio
 - **Adaptadores:** implementações externas (UI, banco, APIs etc.)
- Protege o núcleo (domínio) das mudanças externas, facilitando integrações e testes.



Arquitetura Hexagonal (Ports and Adapters)



Arquitetura Hexagonal

Arquitetura Hexagonal é um modelo de arquitetura que busca **separar as regras de negócio das tecnologias externas**, como banco de dados, APIs e interfaces.

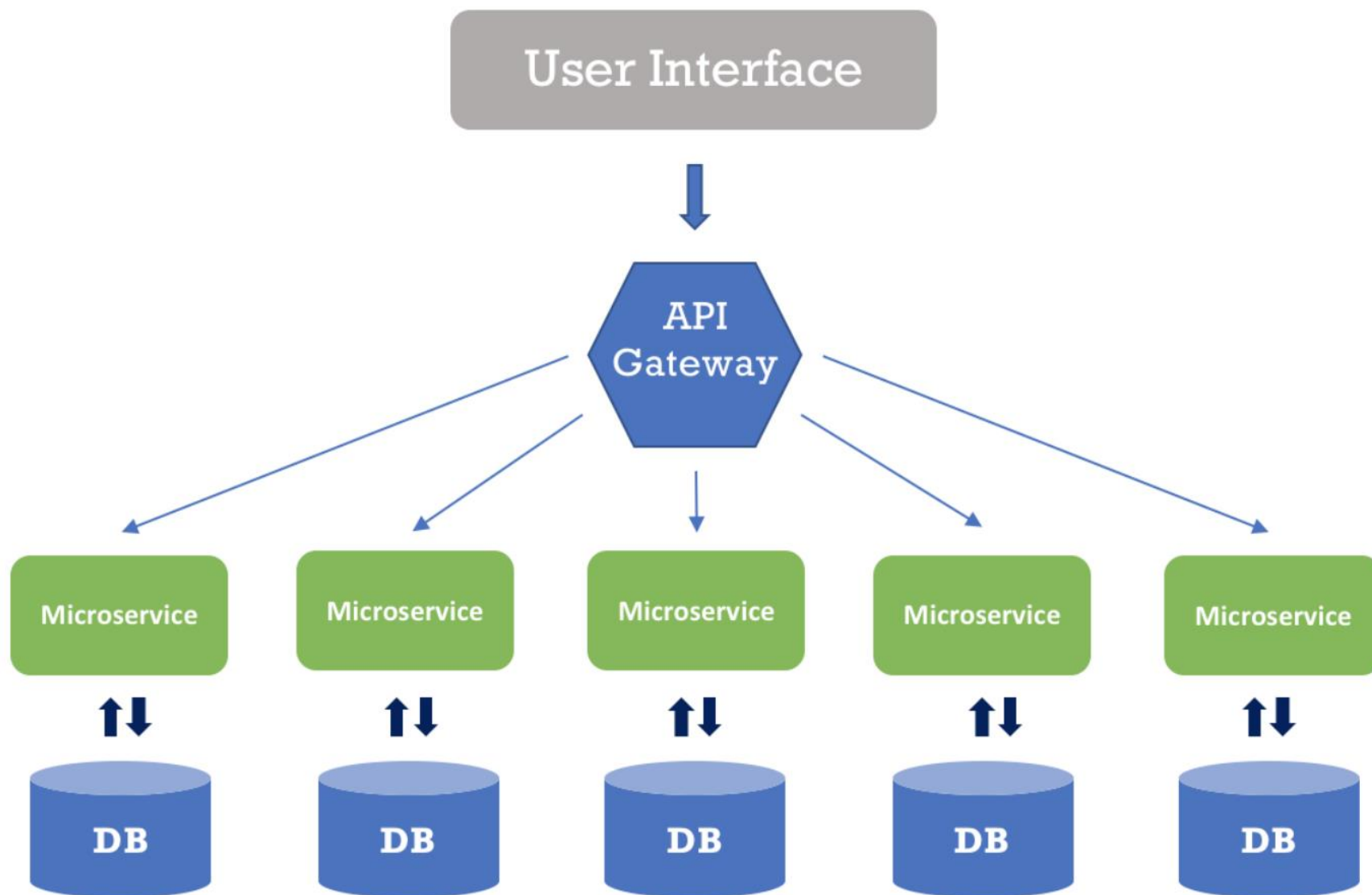
- Ela organiza o sistema em torno do **núcleo da aplicação**, utilizando **portas e adaptadores** para se comunicar com o mundo externo.
 - **Em resumo:**
Regra de negócio no centro → tecnologias ficam nas bordas.
-

Arquitetura de Microserviços

- **Definição:** Estilo arquitetural que divide a aplicação em **pequenos serviços independentes**, cada um responsável por uma funcionalidade específica.
- **Características:**
 - Serviços **autônomos** e **desacoplados**
 - Comunicação via APIs ou mensageria
 - Implantação e escalabilidade independentes
 - Possibilidade de usar tecnologias diferentes em cada serviço



Arquitetura de Microserviços

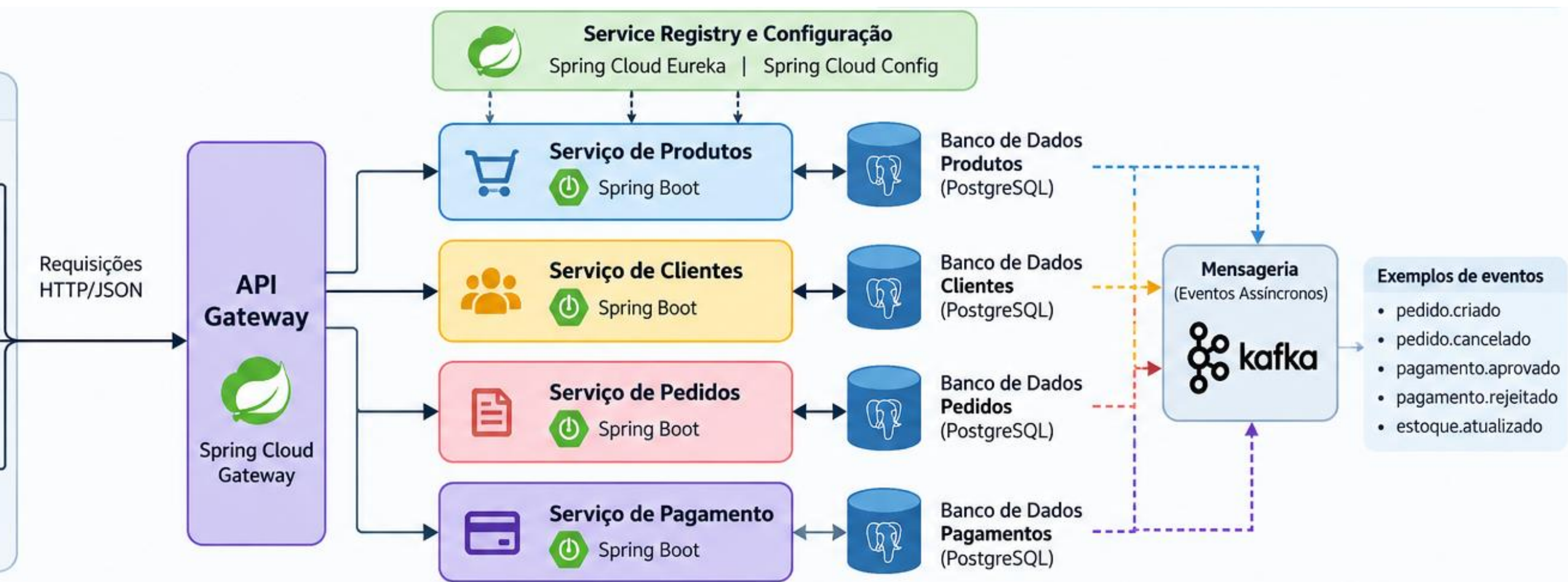


Arquitetura de Microserviços



Principais características

- Serviços independentes e focados em um domínio de negócio
- Cada serviço possui seu próprio banco de dados
- Comunicação via APIs REST ou mensageria (ex.: Kafka, RabbitMQ)
- Implantação e escalabilidade independentes
- Tecnologias podem evoluir de forma isolada



Arquitetura de Microserviços

Cenário prático: Compra de um produto



Resultado: o pedido é confirmado, o pagamento é processado e o estoque é atualizado, tudo de forma desacoplada e escalável.

Arquitetura de Microserviços

- ***Vantagens:***
 - Escalabilidade flexível
 - Facilidade de manutenção e atualização
 - Maior resiliência (falha em um serviço não derruba todo o sistema)
- ***Desafio:***
 - Complexidade na orquestração e monitoramento
 - Exige boa governança de APIs e infraestrutura



Como escolher a arquitetura ideal?

- Analise profundamente o problema: modelo de negócio, requisitos, contexto.
- Observe que arquitetura não é fixa — deve evoluir conforme o projeto.
- Evite seguir modismos sem avaliar adequação ao cenário.



Pense a respeito

- Arquitetura de software é um sistema vivo, que exige estudo e adaptação contínua.
- O uso de padrões auxilia na construção de soluções robustas e eficientes, com melhor comunicação entre equipes.
- A escolha da arquitetura deve ser estratégica, alinhada ao domínio do negócio, e aproveitando boas práticas (como princípios de design) para fortalecer o resultado final.



🔥 Respiração do Código, Forma 2!

ATÉ A PRÓXIMA!

Guardem suas katanas e fechem seus IDEs! Lembrem-se:

Nenhum bug resiste a um bom refactoring e aos testes automatizados.

🐛 Sem Bugs

🧹 Clean Code

🎓 NIFEF



Everyone, lets have fun after this!

🦋 "Mantenha o coração queimando e o código compilando!" 🦋